

Laboratory work 8
Streams and files.
Work with files

Objective: to learn to use text files and work with them using the basic functions of the C language file system.

Short theoretical information

8.1. Streams and files.

In the C I/O system, a single interface is supported for programs, which does not depend on which specific device is being accessed. In this system, there is a generalized input or output device - a stream - between the program and the device. A specific device is called a file.

A file in the C language can be both a disk file and a terminal, printer, etc.

There are two types of streams: text and binary. A text stream is a sequence of characters. The text stream is organized in a line array, each of which ends with a newline character. At the end of the last line, this character is optional.

A binary stream is a sequence of bytes that uniquely corresponds to bytes on the external device. A certain number of zero bytes can be added to the end of a binary stream.

A thread is associated with a specific file by performing an open operation. After opening the file, you can exchange information between the file and the program. If the file can support location requests (pointer of the current position), then when opening such a file, the pointer of the current position in the file is set to the beginning. When reading from the file or writing to it, the pointer of the current position increases, providing movement through the file. A file is detached from a specific thread using the close operation. When a file opened for output is closed, the contents (if any) of its associated stream are written to an external device (this process is called overwriting the stream). If the program terminates normally (the main() function returns control to the operating system or the exit() function is called), then all files are closed automatically. In the event of an emergency termination of the program (program crash or termination by calling the abort() function), the files are not closed.

Note. The exit() function causes an immediate normal termination of the program. The abort() function causes the program to crash immediately.

8.2. The main functions of the file system of the C language.

The main functions of the C language file system are presented in the table.

8.1. To work with these functions, it is necessary to connect the header file <stdio.h> to the program code.

Table 8.1 – Basic functions of the C language file system

Name	Purpose
fopen()	opens a file
fclose()	closes the file
putc()	writes a character to a file
fputc()	writes a character to a file
getc()	reads a character from a file
fgetc()	reads a character from a file
fgets()	reads a line from a file
fputs()	writes a string to a file
fprintf()	for a file is the same as printf() for the console
fscanf()	for a file is the same as scanf() for a console
fseek()	sets the current position pointer to a specific one file byte
ftell()	returns the current pointer value of the current position in the file
feof()	returns true if reached end of file
ferror()	returns true if an error occurred
rewind()	sets the pointer of the current position to the beginning file
remove()	deletes the file
fflush()	to write the stream to the file

Each thread associated with a file has a control structure that is of type FILE and contains information about the file (eg, its name, status, current position pointer to the beginning of the file). A file pointer is a pointer to a FILE type structure. In order to perform reading and writing operations in a file, the program must use the pointer of the corresponding file. An example of declaring a file pointer variable:

```
FILE *fp;
```

Opening the file. The fopen() function opens a stream, associates a specific file with this stream, and returns a pointer to this file. The fopen() function prototype is as follows:

```
FILE *fopen(const char *file_name, const char *mode);
```

where filename is a pointer to a character string that is a valid file name that can include a path specification for that file. The string pointed to by the mode determines how the file will be opened. The file can be opened either in one of the text or one of the binary modes. In the table 8.2 shows the permissible values of the mode line.

Table 8.2 – Valid values of the mode string

Regime	Value
r	open the text file for reading
w	create a text file for writing (if the file exists, its contents will be overwritten)
a	add to the end of the text file (if the file does not exist, it will be created)
rb	open a binary file for reading
wb	create a binary file for writing (if the file exists, its contents will be overwritten)
ab	append to the end of the binary file (if the file does not exist, it will be created)
r+	open a text file for reading/writing
w+	create a read/write text file (if the file exists, its contents will be overwritten)
a+	to the end of a text file or create a read/write text file
r+b	open binary file for reading/writing
w+b	create a read/write binary file (if the file exists, its contents will be overwritten)
a+b	add to the end of a binary or create a read/write binary

Strings like "r+b" can be represented as "rb+".

If an error occurs when opening a file, then `fopen()` returns an empty (null) pointer.

Examples of opening a file for writing:

```
FILE *fp;  
fp = fopen("test.txt", "w");  
or  
FILE *fp;  
if ((fp = fopen("test.txt", "w"))==NULL)  
{  
    printf("Error opening file.\n");  
    exit(1);  
}
```

The maximum number of simultaneously open files is at least 8.

Closing the file. The `fclose()` function closes the stream that was opened by calling `fopen()`, writes all data remaining in the disk buffer to a file, and closes the file at the operating system level. The `fclose()` function prototype is as follows:

```
int fclose(FILE *pf);  
where pf is the file pointer returned as a result of the fopen() call.
```

A return of zero means a successful close operation, in case of an error EOF is returned.

Record character. Two equivalent functions are defined for character output: `putc()` and `fputc()`. The `putc()` function writes a character to a file opened with the

fopen() function in write mode. The prototype of this function is as follows:

```
int putc(int ch, FILE *pf);
```

where pf is the file pointer returned by the fopen() function, and ch is the character to be output (ch is defined as an int, but only the least significant byte is written). If the putc() function was executed successfully, then the written character is returned, otherwise EOF is returned.

Reading the symbol. There are also two equivalent functions for character input: getc() and fgetc(). The getc() function reads a character from a file opened with the fopen() function in read mode.

The prototype of this function is as follows:

```
int getc(FILE *pf);
```

where pf is the file pointer returned by the fopen() function. The getc() function returns an integer value, but the character is in the least significant byte. If the end of the file is reached or in case of an error, the function getc() returns EOF.

Using feof(). The feof() function determines whether the end of the file has been reached. Its prototype is as follows:

```
int feof(FILE *pf);
```

If the end of the file is reached, feof() returns true, otherwise - zero. The following code will read the file until the end of the file is reached:

```
while(!feof(fp)) ch = getc(fp);
```

String input/output: fputs() and fgets(). Function

fgets() reads character strings from a file on disk, function

fputs() writes strings of characters to a file on disk. The fgets() and fputs() function prototypes are as follows:

```
int fputs(const char *line, FILE *pf);
```

```
char *fgets(char *line, int length, FILE *pf);
```

The fputs() function writes the string it points to in a given stream line. On error, this function returns EOF.

The fgets() function reads a string from a given stream until there is no newline character read or number of characters read will not equal length-1. If a line separator was read, it is written to the line that distinguishes the fgets() function from the gets() function. The resulting string will end with the end-of-string character ('0'). Upon successful completion, the function returns a string, and in the event of an error, an empty pointer (null).

Functions fread() and fwrite(). For reading and writing data, the type of which can occupy more than 1 byte, in the file system of the C language there are two functions: fread() and fwrite(). These functions allow you to read and write data blocks of any type. Their prototypes are as follows:

```
size_t fread(void *buffer, size_t few_bytes,
```

```
size_t counter, FILE *pf);
```

```
size_t fwrite(const void *buffer, size_t
```

```
how many_bytes, size_t counter, FILE *pf);
```

For fread(), the buffer is a pointer to an area of memory into which data will be read from the file. For fwrite(), the buffer is a pointer to the data that will be written to the file. The value of the counter determines how many data elements

are read or written, and the length of each element in bytes is equal to `kilk_bytes`.

The `fread()` function returns the number of elements read.

If the end of file is reached or an error occurs, the return value may be less than the counter.

The `fwrite()` function returns the number of elements written.

If no error occurred, the returned result will be equal to the counter value.

Functions `fprintf()` and `fscanf()`. The functions `fprintf()` and `fscanf()` are similar to `printf()` and `scanf()`, but are designed to work with files. The prototypes of these functions are as follows:

```
int fprintf(FILE *pf, const char *control_line, ...);
```

```
int fscanf(FILE *pf, const char *control_line, ...);
```

Note. Because ASCII data is written as it should appear on the screen (rather than in binary), calls to these functions incur some overhead. Therefore, if file size or speed is important, you should use the `fread()` and `fwrite()` functions.

Direct access input/output: `fseek()` function. With direct access, I/O can be performed using the `fseek()` function, which sets a pointer to the current position in the file. The prototype of this function is as follows:

```
int fseek(FILE *pf, long int how_many_bytes, int by_start_count);
```

where `pf` is the file pointer returned as a result of the call to the `fopen()` function, `how_many_bytes` is the number of bytes, starting from `count_start`, that defines the new value of the current position pointer.

In order to access a file at a distance of `several_bytes` from the beginning of the file, the `start_count` must be equal to `SEEK_SET`; so that when accessing, the distance is measured from the current position - `SEEK_CUR`; from the end of the file - `SEEK_END`. The `fseek()` function returns zero on success, and a non-zero value on error.

Direct access is usually used for binary files, because text files may undergo character conversions, so there may be a mismatch between the contents of the file and the byte being accessed. For a text file, the `fseek()` function can be used to access the position found by `ftell()`.

The `ftell()` function determines the current value of the pointer at the exact position in the file. The prototype of this function is as follows:

```
long int ftell(FILE *pf);
```

In case of failure, the function returns -1.

The `rewind()` function. The `rewind()` function sets the pointer of the current position in the file to the beginning of the file. Its prototype is as follows:

```
void rewind(FILE *pf);
```

where `pf` is a valid file pointer.

Function `ferror()`. The `ferror()` function determines whether an error occurred while performing a file operation. Prototype of this function following:

```
int ferror(FILE *pf);
```

This function returns true if an error occurred during the last file operation; otherwise it returns false.

The `remove()` function. The `remove()` function deletes the specified file. Its

prototype is as follows:

```
int remove(const char *file_name);
```

This function returns null on success, otherwise is a non-zero value.

`fflush()` function. The `fflush()` function is used to overwrite the contents of the output stream to a file. Its prototype is as follows:

```
int fflush(FILE *pf);
```

This function writes all the data in the buffer to the file specified by `pf`. When the `fflush()` function is called with an empty (null) pointer to the `pf` file, overwriting will be performed in all files open for output. `fflush()` returns null on success, otherwise EOF.

8.3. Entering data into a text file.

To perform this task, you must first declare a file pointer variable:

```
FILE *fp;
```

Next, you need to open a stream and associate that stream with a specific file by opening a text file in write mode as described above, for example:

```
char name[50]; //file name fp = fopen(name, "w");//open file for writing
```

After the file is opened, you can start recording data using the functions described above, for example:

```
char str[80];
```

```
fputs(str, fp); // write a string of characters to a file on the disk
```

After completing operations of outputting data to the file, it must be closed.

A file is detached from a specific thread:

```
fclose(fp);
```

8.4. Reading data from a text file.

To perform this task, as well as for the task of entering data into a file, you must first declare a file pointer variable. Next, you need to open a stream and associate that stream with a specific file by performing an operation to open a text file in read mode, for example:

```
fp = fopen(name, "r");//opening the file for reading
```

After the file is opened, you can start reading the data using the functions described above. Data is read until the end of the file is reached. Example:

```
while(!feof(fp)) ch = getc(fp); // read data until end of file is reached
```

To read and display all lines of the file on the screen, you can use, for example, the following code:

```
while (fgets(str, 80, fp)) printf(str);
```

After the data reading operations are completed, it must be closed
file

Note. Using the following code to read and

outputting file lines to the screen is incorrect because it duplicates output of the last line of the file. If the file contains, for example, 10 bytes, and 10 bytes are read from the file, `feof()` will return 0 because, although the file pointer is at the end of the file, no attempt was made to read beyond the end of the file. Only after

attempting to read the 11th byte will feof() return a non-zero value.

```
while (!feof(fp)) { //Wrong usage
    fgets(str, 80, fp);
    printf(str);
}
```

Example 8.1.

Write 10 sentences entered from the keyboard into a text file. Read this file and display its contents.

Program text

```
#define _CRT_SECURE_NO_WARNINGS
#define S 80
#define F 50
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
void write_file(char name[F]);
void read_file(char name[F]);
int main(void)
{ SetConsoleCP(1251);
  SetConsoleOutputCP(1251);
  char fname[F];
  printf("Enter a file name:\n");
  scanf("%s%c", &fname);
  write_file(fname);
  read_file(fname);
  return 0;
}
void write_file(char name[F])
{
  char str[S];
  FILE *fp;
  if ((fp = fopen(name, "w")) ==
      NULL) {
    printf("An error occurred while opening the file.\n");
    exit(1);
  }
  for (int i = 0; i < 10; ++i)
  {
    gets(str);
    strcat(str, "\n");
    fputs(str, fp);
  }
  fclose(fp);
}
```

```

}
void read_file(char name[F])
{
char str[S];
FILE *fp;
if ((fp = fopen(name, "r")) ==
NULL) {
printf("An error occurred while opening the file.\n");
exit(1);
}
while (fgets(str, S, fp))
printf(str);
fclose(fp);
}

```

Note. An asterisk (*) before the field format specifier letter in the control line of the `scanf()` function serves to suppress entering redundant data. The `SetConsoleCP()` function sets the input code page used by the console associated with the calling process. The `SetConsoleOutputCP()` function sets the output code page used by the console associated with the calling process. A code page is a table that assigns a certain symbol (or its absence) to each byte value.

For encoding texts in the Ukrainian (Russian) language (that is, Cyrillic letters), one of the most widely used is the Windows-1251 code page (Microsoft code page 1251, CP1251 for Windows systems).

To work with the `SetConsoleCP ()` and `SetConsoleOutputCP ()` functions, it is necessary to connect to the program code header file `<windows.h>`.

Example 8.2.

The following example solves the problem from Example 8.1 using the `rewind()` function. In this example, a text file is opened in read/write mode, and the contents of the file are displayed immediately after it is created.

Program text

```

#define _CRT_SECURE_NO_WARNINGS
#define S 80
#define F 50
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
void wr_file(char name[F]);
int main(void)
{
SetConsoleCP(1251);
SetConsoleOutputCP(1251);
char fname[F];
printf("Enter the file name:\n");

```

```

scanf("%s%c", &fname);
wr_file(fname);
return 0;
}
void wr_file(char name[F])
{
char str[S];
FILE *fp; /* file pointer variable declaration*/
if ((fp = fopen(name, "w+")) == NULL) /* creating a text read/write file*/
{
printf("Error opening file.\n");
exit(1);
}
for (int i = 0; i < 10; ++i)
{
get(str);
strcat(str, "\n"); /* add line separator */
fputs(str, fp); /* write a string of characters to the file on disk*/
}
rewind(fp); /* set the pointer of the current position to the beginning of the file*/
while (fgets(str, S, fp)) printf(str);
fclose(fp); /* close the file*/
}

```

Tasks of laboratory work

Task 1

1. Create a text file and write ten numbers in it, find their sum and display it on the screen.
2. Create a text file and write the sentence "I am a student of Khmelnytskyi National University" into it. Find how many times the letter n occurs in this sentence.
3. Create a text file and write 20 numbers in it. Multiply the second number by 17 and divide by 3. Record the result in a file and display it on the screen.
4. Create a text file and write several sentences in it words, in the next line arrange alphabetically all the letters of this sentence. Save the result to a file and display it on the screen.
5. Create a text file and write the sentence "I am a student of Khmelnytskyi National University" into it. Find how many words a sentence has.

Task 2

1. Create two text files, write 20 in each of them numbers Replace numbers from the first file with numbers from the second and vice versa.
2. Create a text file and write any sentence in it, create another file and rewrite this very sentence into it, but from the end.

3. Create two text files, write 15 in each of them numbers. Multiply the third number from the first file by the fifth number from the second file and divide by the sum of all numbers from the first and second files.

Display the result on the screen.

4. Create a text file and write a 6×7 matrix into it.

Determine the coordinates of the negative elements. Display the coordinates together with the numbers on the screen and in the file.

5. Create a text file and write a 10×9 matrix to it.

Find the sum of elements, both positions of which have an even number. Display the result on the screen.

Task 3.

1. Delete all odd lines in the given text file.

2. A text file is specified. Delete empty lines from it.

3. Given a number k (< 10) and a text file containing more than k lines. Create a new text file containing the last k lines of the source file.

4. Given a string S and a text file. Add line S to beginning file.

5. A text file is specified. Create a text file that contains all characters encountered in the text, including spaces and punctuation marks (without repetitions). Place symbols in descending order of their codes.